Plateforme intelligente de pilotage et d'analyse décisionnelle du CIAT

Rapport de projet

ISEP Amadou Trawaré de Diamniadio · Centre d'Innovation, d'Application et de Transfert
Évaluation pratique en intelligence artificielle générative · Projet 3 · document généré le 03/11/2026

CIAT VISION – Rapport de projet

Plateforme intelligente de pilotage et d'analyse décisionnelle du CIAT
ISEP Amadou Trawaré de Diamniadio · Évaluation pratique en intelligence artificielle générative · Projet 3
Année académique 2026-2027

---

1. Introduction

Le Centre d'Innovation, d'Application et de Transfert (CIAT) de l'ISEP Amadou Trawaré de Diamniadio accompagne des porteurs de projets à travers son incubateur. Pour piloter cette activité, ses responsables doivent suivre des informations nombreuses et dispersées : candidatures, évaluations, projets incubés, jalons, cohortes, financements, partenariats. Sans outil dédié, produire un indicateur fiable (« combien de projets sont en retard ? ») demande un travail manuel long et source d'erreurs.

CIAT VISION répond à ce besoin. C'est une application web décisionnelle qui centralise les données dans une base PostgreSQL, calcule automatiquement des indicateurs clés de performance, les présente dans des tableaux de bord interactifs, produit des rapports, et permet d'interroger les données en langage naturel grâce à un assistant fondé sur l'intelligence artificielle générative.

Problématique. Comment mettre à la disposition des responsables du CIAT une solution numérique qui centralise les données, suit les performances des projets, analyse l'activité de l'incubateur et facilite la décision grâce à l'IA générative, **sans jamais présenter un chiffre inexact** ?

Cette dernière exigence a guidé l'ensemble de la conception : un outil décisionnel qui « hallucine » une statistique est plus dangereux qu'utile.

2. Démarche

Le développement a suivi la méthode progressive imposée par le sujet, en douze phases, chacune donnant lieu à un livrable vérifiable :

Phase	Livrable	Document
1. Analyse des besoins	Cahier des charges (v1.2)	`01_cahier_des_charges.md`
2. Architecture	Diagrammes UML (cas d'utilisation, classes, séquences, états, déploiement)	`02_architecture.md`
3. Base de données	Modèle entité-association, dictionnaire des données	`03_base_de_donnees.md`
4. Maquettes	23 écrans (ordinateur et téléphone)	`04_maquettes.md`
5 à 10. Réalisation	Backend, frontend, IA, rapports, importation	code source
11. Tests	88 tests automatisés, journal de 10 anomalies	`05_tests_et_anomalies.md`
12. Déploiement	Blueprint Render, guides	`06`, `07`, `render.yaml`

L'IA générative (Claude, dans l'environnement Claude Code) a été utilisée comme assistant de développement ; chaque échange est consigné dans le **journal des prompts**. Son apport le plus utile n'a

pas été la production de code, mais la **revue critique** : la relecture du cahier des charges 1.0 a fait apparaître des manques qui auraient bloqué la suite (aucune définition d'un « projet en retard », aucun statut pour une candidature rejetée, droits de modification non attribués, interdiction du SQL arbitraire non reprise). Ces points ont été tranchés avant de modéliser la base.

3. Analyse des besoins

3.1. Utilisateurs et droits

Trois profils ont été retenus. L'**administrateur** gère les comptes, les paramètres et consulte le journal. Le **coordonnateur** pilote l'activité : il crée et met à jour projets, cohortes, évaluations et financements, et importe des données. L'**analyste** consulte, analyse et exporte, sans modifier les données ni voir les coordonnées des porteurs. La matrice complète figure au § 4.2 du cahier des charges ; elle est appliquée **côté serveur**.

3.2. Règles de gestion

Le cycle de vie d'un projet a été formalisé en un diagramme d'états : *En attente* → *En évaluation* → *Sélectionné* → *En incubation* → *En développement* → *Prototype réalisé* → *Terminé*, avec les branches *Non retenu* et *Suspendu*. Plusieurs règles en découlent : une candidature ne peut être acceptée ou rejetée qu'après au moins une évaluation ; le passage en incubation crée automatiquement cinq jalons ; « Terminé » impose 100 % de progression ; une baisse de progression doit être justifiée ; une suspension n'est possible qu'à partir de l'incubation.

3.3. Indicateurs

Chaque indicateur a reçu une définition précise, gérant les dénominateurs nuls :

Indicateur	Définition
Taux de sélection	candidatures acceptées ÷ (acceptées + rejetées)
Taux d'achèvement	projets terminés ÷ projets incubés
Taux de retard	projets en retard ÷ projets actifs
Projet en retard	projet actif dont la date limite est dépassée **ou** dont un jalon non atteint a une date prévue dépassée (au-delà d'un seuil de tolérance paramétrable)
Financement total	somme des montants **reçus** (l'accordé est affiché à côté)

4. Conception

4.1. Architecture

L'application suit une architecture en trois couches. Le **frontend** (React 19, TypeScript, Vite, Tailwind CSS, Recharts, Lucide) ne contient aucune donnée ni aucun calcul : il affiche ce que renvoie l'API. Le **backend** (FastAPI, SQLAlchemy 2, Pydantic 2) porte l'authentification, les droits, les règles métier, le moteur d'indicateurs, les rapports, l'importation et l'assistant. La **base PostgreSQL 16** stocke les données.

Un principe structurant : **chaque indicateur est défini une seule fois** (`backend/app/services/kpi.py`). Le tableau de bord, les graphiques, les rapports PDF/Excel/CSV et l'assistant IA appellent ces mêmes fonctions. Les chiffres sont donc identiques partout, et un test automatique le vérifie.

4.2. Base de données

Seize tables : les douze demandées (`users`, `projects`, `project_leaders`, `domains`, `cohorts`, `applications`, `evaluations`, `milestones`, `fundings`, `partners`, `activities`, `audit_logs`) et quatre tables techniques (historique des statuts, historique de l'assistant, paramètres, traçabilité des imports). Les règles importantes sont **aussi** garanties par des contraintes `CHECK` en base (statuts autorisés, `reçu ≤ accordé ≤ demandé`, ordre des dates, 100 % si terminé) : même un script mal écrit ne peut pas enregistrer une donnée incohérente. Le schéma est versionné par une migration Alembic ; le dictionnaire des données est **généré à partir du code**, ce qui garantit qu'il reste exact.

4.3. Interface

L'interface respecte la palette de l'ISEP (marron `#4C311F`, beige `#CA9871`, marron profond `#362316`). Une vérification des contrastes a conduit à une règle : **pas de texte blanc sur beige** (contraste 2,5 : 1, illisible) ; les boutons beige portent un texte marron profond (5,9 : 1). Dans les graphiques, une série unique est toujours marron et chaque statut garde la même couleur partout. L'interface s'adapte au téléphone : grilles qui se réorganisent, tableaux transformés en cartes, menu coulissant, filtres repliables.

5. Réalisation

Module	Réalisation
1. Authentification	JWT (60 min, renouvelé si l'utilisateur est actif), mots de passe bcrypt, blocage 15 min après 5 échecs, même message pour un identifiant inconnu ou un mauvais mot de passe
2. Tableau de bord	8 KPI cliquables, 4 taux, filtres partagés (période, cohorte, domaine, statut)
3. Projets	Recherche, filtres, tri, pagination, création avec porteur, transitions contrôlées, progression, évaluations, jalons, suppression logique
4. Analyses	Les 9 analyses du prompt + 3 analyses financières ; graphiques en barres, anneau, courbes, aires
5. Cohortes	Création, modification, comparaison multi-cohortes, détail (projets, activités)
6. Financements	Budget, demandé, accordé, reçu, reste à recevoir, besoin non couvert ; contrôles de cohérence
7. CIAT AI Analyst	Voir § 6
8. Rapports	6 rapports en PDF (logo, période, KPI, graphiques vectoriels, commentaire, date, mention « simulées »), Excel et CSV
9. Administration	Comptes, rôles, paramètres, journal d'audit, purge des données simulées, restauration
10. Importation	Modèles, contrôle des colonnes, formats, valeurs, doublons ; prévisualisation ; confirmation ; transaction unique

Les **données simulées** (269 candidatures, 4 cohortes et la cohorte suivante en cours de sélection, 8 domaines) sont générées de façon reproductible et cohérente dans le temps : les retards apparaissent naturellement lorsque des jalons ne sont pas atteints. Chaque enregistrement porte l'indicateur

`is_simulated` ; un bandeau, les rapports et l'assistant le rappellent. L'administrateur peut purger ces données en une action, puis importer les données réelles.

6. L'intelligence artificielle générative : CIAT AI Analyst

6.1. Le risque à maîtriser

Un modèle de langage peut produire un chiffre plausible mais faux. Laisser le modèle écrire lui-même des requêtes SQL (approche « texte vers SQL ») expose en outre à des requêtes erronées, coûteuses ou dangereuses, et lui donne accès aux données personnelles. Le prompt de cadrage interdit d'ailleurs explicitement le SQL arbitraire.

6.2. La solution retenue : appel d'outils sur une liste fermée

Le modèle (Claude Opus 5.5, via l'API d'Anthropic) reçoit la question, le référentiel (noms exacts des cohortes, domaines, statuts) et la description de **sept fonctions d'analyse** : indicateurs clés, répartition des projets, projets en retard, comparaison des cohortes, synthèse des financements, projets à surveiller, évolution des candidatures. Il ne peut **que** demander l'exécution de l'une d'elles. Le backend : 1. vérifie que la fonction figure dans la liste ; 2. valide les paramètres (schéma strict) et les traduit en filtres ; 3. exécute la fonction, qui réutilise le moteur d'indicateurs ; 4. renvoie au modèle un résultat **agrégé, sans aucune donnée personnelle** ; 5. en cas de paramètre erroné, renvoie un message d'erreur que le modèle utilise pour se corriger.

Le modèle rédige alors la réponse à partir de ces seuls résultats, selon un format imposé : réponse directe, chiffres utilisés, recommandations clairement distinguées, limites. L'interface affiche les fonctions appelées et leurs résultats bruts : **chaque chiffre est vérifiable**. Un repli automatique sur un autre modèle est activé si une requête est déclinée.

6.3. Robustesse

Sans clé d'API, ou si le service est indisponible, l'assistant passe en **mode dégradé** : une reconnaissance de mots-clés appelle la même fonction d'analyse et des gabarits mettent en forme les chiffres. Les huit questions d'exemple du cahier des charges sont reconnues (test automatique). L'historique est conservé, une limite de questions par jour maîtrise les coûts, et l'IA sert aussi à **proposer** le commentaire des rapports, toujours relu par l'utilisateur avant export.

6.4. Ce qui a été vérifié

La boucle d'appel d'outils a été testée avec un client Claude simulé (appel de fonction, transmission des résultats, erreur renvoyée au modèle). Le comportement du modèle réel n'a pas pu être évalué faute de clé d'API dans l'environnement de développement : c'est la principale vérification restant à faire (§ 8).

7. Sécurité et protection des données

Mots de passe hachés (bcrypt) et politique de complexité ; jetons signés à durée limitée ; contrôle des rôles sur chaque route ; validation de toutes les entrées (Pydantic) et contraintes en base ; requêtes paramétrées par l'ORM (pas d'injection SQL) ; en-têtes de sécurité HTTP ; CORS restreint ; secrets dans les variables d'environnement ; journal d'audit de toutes les actions sensibles ; suppression logique des

projets. Les données des porteurs sont des données personnelles au sens de la **loi sénégalaise n° 2008-12** : elles ne sont jamais transmises au service d'IA, et leurs coordonnées sont masquées pour le rôle Analyste.

8. Tests et résultats

88 tests automatisés réussissent : 80 pour le backend, exécutés sur une vraie base PostgreSQL de test, et 8 pour le frontend. Le test le plus important **recalcule chaque indicateur indépendamment**, en Python, à partir des lignes brutes, et le compare au moteur SQL, sans filtre et pour chaque cohorte, domaine, statut et période. Un parcours automatisé dans Microsoft Edge a visité les 12 écrans à 1 440 px et 390 px : aucune erreur JavaScript, aucun défilement horizontal sur téléphone.

Ce parcours a révélé l'anomalie la plus sérieuse : la page de l'assistant **plantait après la première réponse**. La cause était subtile : un effet React renvoyait le résultat de `scrollIntoView()`, une promesse dans les navigateurs récents, que React prenait pour une fonction de nettoyage. Elle a été corrigée et tous les effets similaires ont été harmonisés. Dix anomalies ont été consignées avec leur cause, leur correction et leur vérification.

Performances mesurées sur une base de **1 030 projets**, la volumétrie de la cible : tableau de bord complet affiché dans le navigateur en 1,0 à 1,8 s (cible : < 2 s), indicateurs en 38 ms, rapport PDF le plus lourd (1 030 lignes) en 4,8 s (cible : < 10 s).

9. Limites et perspectives

Limite	Perspective
Données réelles indisponibles : les chiffres présentés sont simulés	Importer les données du CIAT (procédure documentée) et valider les définitions des KPI avec la direction
Mode IA non évalué avec le modèle réel	Constituer un jeu de 20 questions de référence et vérifier chaque chiffre cité (critère § 16.2)
Performances mesurées en local, sur 1 030 projets	Les mesurer sur l'hébergement définitif ; paginer le rapport PDF des projets au-delà de quelques milliers de lignes
Offres d'hébergement gratuites (mise en veille, durée de vie de la base)	Hébergement institutionnel ou offre payante, sauvegardes planifiées
Réalisation individuelle d'un sujet prévu pour trois	Organisation à valider avec l'enseignant

Évolutions possibles : dépôt de candidature en ligne par les porteurs, notifications de retard par e-mail, indicateurs d'impact (emplois créés, chiffre d'affaires), authentification à deux facteurs.

10. Conclusion

CIAT VISION est une application fonctionnelle et testée qui couvre les dix modules demandés. Son intérêt principal tient à sa **fiabilité** : des indicateurs définis sans ambiguïté et calculés en un seul endroit, des règles métier garanties jusque dans la base, et un assistant d'IA générative conçu pour ne citer que des chiffres vérifiables. Le projet montre que l'IA générative apporte une réelle valeur à un outil décisionnel à

condition d'être encadrée : le modèle interprète la question et rédige la réponse, mais ce sont des fonctions contrôlées qui produisent les chiffres.